

PW EARNERS
DSA in C++

Daily Practice Problems (DPP)

Topic: Conditionals (If-Else, Logical Operators, Ternary, Switch)

For Q1 to Q10: Predict the exact output of each program block. If there is a compilation error or undefined behavior, state the reason clearly.

Q1.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 5, b = 10;
5     if(a == b) {
6         cout << "Equal matches " << a << endl;
7     } else {
8         cout << "Not equal " << a << endl;
9     }
10    return 0;
11 }
```

Output:

.....

Q2.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 5, b = 10;
5     if(a = b) {
6         cout << "Equal matches " << a << endl;
7     } else {
8         cout << "Not equal " << a << endl;
9     }
10    return 0;
11 }
```

Output:

.....

Q3.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int num = -7;
5     if (num >= 0) {
```

```
6         cout << "Inside Positive block" << endl;
7     } else {
8         cout << "Inside Negative block" << endl;
9     }
10    return 0;
11 }
```

Output:

.....

Q4. (Concept: Short-circuit evaluation)

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4     int x = 0, y = 5;
5     if(x != 0 && (y = y + 10) > 10) {
6         cout << "Inside if branch" << endl;
7     }
8     cout << "x = " << x << ", y = " << y << endl;
9     return 0;
10 }
```

Output:

.....

Q5.

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4     int p = 1, q = 20;
5     if(p == 1 || (q = q + 5) > 25) {
6         cout << "First branch hit" << endl;
7     }
8     cout << "p = " << p << ", q = " << q << endl;
9     return 0;
10 }
```

Output:

.....

Q6.

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4     int marks = 45;
5     bool pass = (marks >= 40) ? true : false;
6     int bonus = pass ? 5 : 0;
7     cout << "Pass status: " << pass << ", Bonus: " << bonus << endl;
8     return 0;
9 }
```

Output:

.....

Q7.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 12, y = 8;
5     int max_val = (x > y) ? x : y;
6     cout << "Maximum value is " << max_val << endl;
7     return 0;
8 }
```

Output:

.....

Q8. (*Concept: Switch fall-through effect*)

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int code = 2;
5     switch(code) {
6         case 1: cout << "Alpha" << endl;
7         case 2: cout << "Beta" << endl;
8         case 3: cout << "Gamma" << endl;
9                 break;
10        default: cout << "Omega" << endl;
11    }
12    return 0;
13 }
```

Output:

.....

Q9.

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int val = 65;
5     switch(val) {
6         case 'A': cout << "Character match A" << endl; break;
7         case 65: cout << "Integer match 65" << endl; break;
8         default: cout << "No match found" << endl;
9     }
10    return 0;
11 }
```

Output:

.....

Q10. (*Crazy Question*)

```

1  #include <iostream>
2  using namespace std;
3  int main(){
4      int a = 0, b = 0;
5      if (a++ && ++b) {
6          cout << "Condition standard check true" << endl;
7      }
8      cout << "a = " << a << ", b = " << b << endl;
9      return 0;
10 }
```

Output:

.....

For Q11 to Q20: Write clean, modular, and compilable C++ programs. Ensure proper use of data types, validation where necessary, and formatted user prompts.

Q11. Take two integers as input from the user. Determine and print whether their multiplication product is positive, negative, or zero without calculating the absolute numeric valuation value if possible.

Sample — Input: -4 5 $\Rightarrow$ Output: Product is Negative

Q12. Take a positive integer input from the user and check if it is divisible by both 3 and 5. Do **not** use the logical AND (&&) operator inside your conditional test statement. (*Hint: Think about how mathematical properties like the Lowest Common Multiple (LCM) can reduce multi-step rules*).

Sample — Input: 30 $\Rightarrow$ Output: Divisible by both 3 and 5

Q13. Given the number of sides n of a simple polygon and the sum of all its interior angles S , write a program to check if the user has provided valid dimensional specifications. Use the structural formula:

$$S = (n - 2) \times 180^\circ$$

Sample — Input: Sides = 3, Sum = 180 $\Rightarrow$ Output: Valid Polygon

Q14. Take two integers as input. If exactly one of them is odd, print "One is odd". If both are odd, print "Both are odd". If neither of them is odd, print "None is odd". Use conditional constructs effectively.

Sample — Input: 4 7 $\Rightarrow$ Output: One is odd

Q15. Write a menu-driven application using a C++ **switch-case** structure that accepts an integer number representing a month (1 to 12). Print the corresponding month's standard calendar name. If the input falls outside the standard scope, default cleanly with an error flag message.

Sample — Input: 2 $\Rightarrow$ Output: February

Q16. (*Concept Booster: Upper/Lower Boundary constraints*)

Take a character input from the keyboard. Write a conditional structure to evaluate whether the provided letter is an uppercase English alphabet (*A-Z*), a lowercase alphabet (*a-z*), a numerical digit (0-9), or a special symbolic character.

Sample — Input: g $\Rightarrow$ Output: Lowercase Alphabet

Q17. (*Concept Booster: Leap Year Calculation Rule*)

Write a program that takes a positive integer representing a year (e.g., 2024) and checks whether it is a leap year or not. Your logical conditions must accurately check for standard four-year intervals while handling century-year bounds correctly (100 and 400 rules).

Sample — Input: 1900 $\Rightarrow$ Output: Not a Leap Year

Q18. (*Concept Booster: Coordinate System Analysis*)

Given the three coordinate points $A(x_1, y_1)$, $B(x_2, y_2)$, and $C(x_3, y_3)$ as floating-point inputs, write a program to determine if all three points sit sequentially on the same straight line (collinear points). Use the slope comparison formulation:

$$(y_2 - y_1) \times (x_3 - x_2) == (y_3 - y_2) \times (x_2 - x_1)$$

Sample — Input: 0 0 2 2 4 4 $\Rightarrow$ Output: Points are Collinear

Q19. (Concept Booster: Triangle Categorization Code)

Take three numerical values denoting the side dimensions (a, b, c) of a shape. First, use a validation structure to test whether these segments can structurally assemble into a valid triangle ($a + b > c$, $b + c > a$, and $c + a > b$). If valid, further categorize the object as an **Equilateral**, **Isosceles**, or **Scalene** triangle.

Sample — Input: 5 5 8 $\Rightarrow$ Output: Isosceles Triangle

Q20. (Concept Booster: Electric Bill Multi-Tier Grid Pricing)

An energy cooperative charges electricity rates dynamically depending on consumer usage units. Write a program to compute total charges based on the input consumption units:

- First 100 units: \$1.50 per unit
- Next 200 units (101 to 300): \$2.50 per unit
- Above 300 units: \$4.00 per unit

An additional structural safety surcharge of 15% is appended to the total amount generated. Find the net bill payable.

Sample — Input: 150 units $\Rightarrow$ Output: Total Bill = \$316.25

All the best! Keep Coding ♡